

ADAPTIVE DEPTH AND EXPERT REFINEMENT FOR EFFICIENT SPEECH ENHANCEMENT

Xikun Lu¹Yujian Ma¹Yunda Chen²Xianquan Jiang³Jinqiu Sang^{4*}¹ Shanghai Institute of Artificial Intelligence for Education, East China Normal University, China² Guangdong Key Laboratory of Intelligent Information Processing, Shenzhen University, China³ Boin Hearing Technology (Shanghai) Co., LTD, China⁴ School of Computer Science and Technology, East China Normal University, China

ABSTRACT

Most neural speech enhancement systems use a fixed processing depth for all inputs, which can introduce unnecessary computation when fewer refinement steps are sufficient. We propose *Adaptive Depth and Expert Refinement* (ADER), a parameter-shared progressive enhancement framework with input-dependent computation. ADER combines an *Adaptive Depth Controller* (ADC) for hard early termination with a *Conditional Expert Router* (CER) that selects one lightweight residual adapter at each executed refinement iteration. We further introduce *Exit-aware Intermediate Supervision* (EIS) to directly optimize candidate intermediate outputs for early exit. On VCTK-DEMAND, ADER reduces the parameter count and average computation of MP-SENNet by 70.4% and 51.3%, respectively, while achieving a WB-PESQ of 3.37. Overall, ADER enables input-dependent refinement and reduces redundant computation during inference.

Index Terms— Speech enhancement, adaptive computation, early exiting, parameter sharing, conditional computation.

1. INTRODUCTION

Deep learning has driven substantial progress in monaural speech enhancement (SE), including complex-valued spectral modeling, phase-aware estimation, and time–frequency (TF) sequence modeling [1, 2, 3, 4, 5]. Recent studies have further examined the effects of network architecture, model size, and computational budget on SE performance [6]. As SE systems are increasingly deployed under real-time and resource constraints, computational efficiency has become an important design consideration. A central problem is therefore to reduce inference cost while retaining effective TF modeling for SE.

Existing studies have addressed this problem by designing more efficient SE architectures. FullSubNet combines full-band and sub-band modeling for real-time enhancement [7], while DeepFilterNet uses deep filtering and grouped operations for low-complexity full-band processing [8]. BLOOM-Net further introduces blockwise optimization to produce scalable intermediate enhancement outputs [9]. A different approach is to reduce parameter redundancy through block reuse. Kim *et al.* proposed a progressive SE framework that repeatedly applies a shared processing block instead of stacking independently parameterized blocks [10]. This design enables progressive refinement with substantially fewer independent parameters. However, the repetition depth is predefined and remains fixed

during inference. Consequently, all utterances undergo the same number of refinement iterations, even when fewer iterations may be sufficient for some inputs.

Dynamic computation provides a direct way to adapt the number of executed refinement steps. Adaptive Computation Time introduced input-dependent computational depth in recurrent networks [11], and related ideas have subsequently been applied to speech processing. Li *et al.* introduced early exit into progressive SE using the difference between consecutive enhancement outputs as a stopping criterion [12]. Dynamic nsNet2 incorporates multiple exits into a noise suppression network to vary inference cost across inputs [13]. Early-exit Transformers have also been studied for continuous speech separation [14]. More recently, Olsen *et al.* introduced probabilistic stopping criteria for speech separation and enhancement [15]. These studies demonstrate that processing depth can be adapted to individual inputs. For a block-reuse architecture, however, adaptive depth alone does not address the limited flexibility of full parameter sharing. As refinement proceeds, the latent representation evolves, whereas the same shared block is applied at every iteration. Sparse expert routing can complement the shared block with lightweight state-dependent adaptation [16, 17]. This suggests that block-reuse models should adapt the refinement depth to each input and the refinement operation to the current state.

Inspired by the roles of cellular proliferation and differentiation, which respectively correspond to continuing refinement when needed and applying specialized processing to the current state, we propose *Adaptive Depth and Expert Refinement* (ADER)¹, which combines a shared refinement backbone with adaptive depth control and conditional expert routing. Specifically, our contributions are threefold. 1) We introduce an *Adaptive Depth Controller* (ADC) that determines whether another refinement iteration should be executed, allowing the number of repeated refinement steps to vary across utterances. 2) We introduce a *Conditional Expert Router* (CER) that selects one lightweight residual adapter at each executed iteration through straight-through Top-1 routing. This provides state-dependent adaptation while keeping the main refinement block shared. 3) We introduce *Exit-aware Intermediate Supervision* (EIS), which applies the complete enhancement objective to a randomly sampled intermediate iteration during training, making intermediate outputs more reliable for hard early exit. Experiments on VCTK-DEMAND evaluate the resulting adaptive computation, refinement-depth behavior, and iteration-dependent expert routing.

* Corresponding author: Jinqiu Sang (jqsang@mail.ecnu.edu.cn).

¹Our source codes are available online: <https://github.com/Luxikun669/ADER>

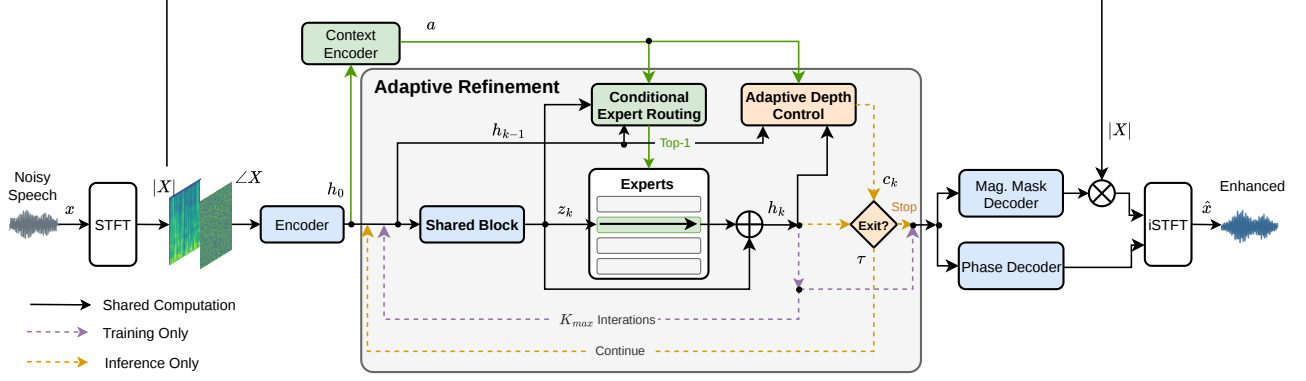


Fig. 1. Overview of the proposed ADER framework. A shared refinement block is repeatedly applied to the encoded TF representation. At each iteration, the CER selects one residual expert adapter, and the ADC determines whether another refinement iteration is required.

2. PROPOSED METHOD

Figure 1 presents the overall architecture of the proposed ADER framework. ADER is built on a parameter-shared progressive refinement backbone, with the CER and ADC controlling the expert transformation and refinement depth, respectively.

2.1. Shared Progressive Refinement

Let $x \in \mathbb{R}^L$ denote a noisy speech waveform with L samples. Following the magnitude–phase front end of MP-SENet [18], its STFT magnitude and phase are processed by the encoder $E(\cdot)$ to obtain an initial TF representation $h_0 \in \mathbb{R}^{C \times T \times F}$, where C , T , and F denote the feature-channel, time-frame, and frequency-bin dimensions, respectively.

The encoded representation h_0 is used in two ways. First, it serves as the initial state of the progressive refinement process. Second, a lightweight context encoder extracts $a = E_{\text{ctx}}(h_0)$, which provides conditioning information to both CER and ADC throughout the refinement process. The context encoder E_{ctx} consists of a 1×1 convolution, instance normalization, and PReLU.

ADER performs progressive refinement using a single TS-Transformer block adopted from MP-SENet [18] as the shared refinement block C_θ . The block models temporal and spectral dependencies in the TF representation and is recursively reused across refinement iterations. At iteration k ,

$$z_k = C_\theta(h_{k-1}), \quad k = 1, \dots, K_{\max}, \quad (1)$$

where h_{k-1} is the current refinement state, z_k is the shared-block output before expert adaptation, and $K_{\max}$ denotes the maximum refinement depth. For $k = 1$, $h_{k-1} = h_0$; subsequent iterations use the refined representation from the preceding iteration. Since the parameters of C_θ are reused across iterations, increasing the refinement depth does not introduce additional copies of the refinement block.

2.2. Conditional Expert Routing

Although the backbone parameters are shared, the representation evolves after each refinement iteration. ADER therefore introduces a lightweight conditional expert branch to adapt the shared-block output according to the current refinement state.

At refinement iteration k , the CER forms its routing descriptor from h_{k-1} , z_k , and the acoustic context a . After global average pooling,

$$q_k^{\text{CER}} = [P(h_{k-1}), P(z_k), P(a)], \quad (2)$$

where $P(\cdot)$ denotes global average pooling and $[\cdot]$ denotes feature concatenation. The absolute-difference term reflects the change introduced by the shared refinement block, while a provides utterance-level context throughout the refinement process.

The router R_{CER} maps q_k^{CER} to an expert probability distribution

$$\pi_k = \text{softmax}\left(\frac{R_{\text{CER}}(q_k^{\text{CER}})}{T_r}\right), \quad (3)$$

where T_r denotes the routing temperature. We use M lightweight expert adapters $\{A_m\}_{m=1}^M$. Each adapter contains a bottleneck 1×1 projection, a depthwise 3×3 convolution, and a 1×1 output projection, together with a learnable residual scale. This design keeps the expert branch small relative to the shared refinement block.

During training, straight-through Gumbel Top-1 routing produces a one-hot routing vector $r_k \in \{0, 1\}^M$. The output of the k -th refinement iteration is

$$h_k = z_k + \sum_{m=1}^M r_{k,m} A_m(z_k). \quad (4)$$

Thus, the shared block is executed at every refinement iteration, while only one expert adapter contributes to the forward output. During inference, the selected expert is $m_k^* = \arg \max_m \pi_{k,m}$, and only $A_{m_k^*}$ is evaluated.

To avoid routing collapse, we apply a Switch-style load-balancing loss independently at each refinement iteration. For expert m , let $p_{k,m}$ denote its mean soft routing probability and $\ell_{k,m}$ its hard selection frequency within the current batch. The balancing term is

$$\mathcal{L}_{\text{bal}} = \frac{M}{K_{\max}} \sum_{k=1}^{K_{\max}} \sum_{m=1}^M \text{sg}(\ell_{k,m}) p_{k,m}, \quad (5)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. Computing this term separately for each iteration encourages routing diversity without requiring different copies of the main refinement block.

2.3. Adaptive Depth Control

Conditional expert routing changes the residual transformation used at each refinement iteration, but it does not determine how many iterations should be executed. We therefore introduce an ADC to decide whether the current representation requires further refinement.

After each refinement iteration, the ADC determines whether further refinement is required using h_{k-1} , h_k , and the acoustic context a . Its input descriptor is

$$q_k^{\text{ADC}} = [P(h_{k-1}), P(h_k), P(a)], \quad (6)$$

$$c_k = \sigma\left(R_{\text{ADC}}(q_k^{\text{ADC}})\right), \quad (7)$$

where R_{ADC} denotes the depth controller, $\sigma(\cdot)$ is the sigmoid function, and $c_k \in [0, 1]$ is the probability of continuing to another refinement iteration.

ADER executes at least $K_{\min}$ and at most $K_{\max}$ refinement iterations, where $K_{\min}$ and $K_{\max}$ denote the minimum and maximum refinement depths, respectively. Once $k \geq K_{\min}$, inference terminates when $c_k < \tau$, where τ is the continuation threshold; otherwise, refinement proceeds to iteration $k+1$. The resulting input-dependent refinement depth is therefore $N(x) \in \{K_{\min}, \dots, K_{\max}\}$.

The ADC requires a training target indicating whether one more refinement iteration is useful. We derive this target from the reconstruction quality along the full refinement trajectory. For the k -th iteration, we use

$$Q_k = 0.9\mathcal{L}_{\text{mag}}^{(k)} + 0.3\mathcal{L}_{\text{pha}}^{(k)} + 0.1\mathcal{L}_{\text{com}}^{(k)}, \quad (8)$$

where $\mathcal{L}_{\text{mag}}^{(k)}$, $\mathcal{L}_{\text{pha}}^{(k)}$, and $\mathcal{L}_{\text{com}}^{(k)}$ denote the magnitude, phase, and complex-spectrum reconstruction losses obtained from the output at iteration k .

Let $Q_{\text{best}} = \min_{j \geq K_{\min}} Q_j$ denote the best reconstruction quality among the valid exit iterations. Rather than forcing every sample to reach the globally best iteration, we select the earliest iteration whose quality is sufficiently close to this minimum:

$$N^* = \min \{k \geq K_{\min} \mid Q_k \leq (1 + \delta)Q_{\text{best}}\}, \quad (9)$$

where δ controls the allowed reconstruction difference from the best iteration. The continuation target is then $t_k = \mathbb{I}[k < N^*]$, where $\mathbb{I}[\cdot]$ is the indicator function.

Only decisions that would be reached before the target exit are included when training the ADC. For sample i , we use the reachability weight $w_{i,k}^{\text{reach}} = \mathbb{I}[K_{\min} \leq k \leq N^*]$. The depth-control objective is

$$\mathcal{L}_{\text{ADC}} = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \frac{\sum_i w_{i,k}^{\text{reach}} \text{BCE}(c_{i,k}, t_{i,k})}{\sum_i w_{i,k}^{\text{reach}} + \epsilon}, \quad (10)$$

where $\mathcal{K} = \{K_{\min}, \dots, K_{\max}-1\}$ is the set of actionable decisions, $\text{BCE}(\cdot)$ denotes binary cross entropy, and ϵ is a small constant for numerical stability.

2.4. Exit-Aware Intermediate Supervision and Training

Reliable intermediate outputs are required for hard early exit. However, under full-depth training, task supervision is applied primarily to the final refinement output, while intermediate states are optimized mainly through subsequent refinement. This creates a mismatch between training and adaptive-depth inference.

To reduce this mismatch, we introduce EIS. During training, ADER always unfolds all $K_{\max}$ refinement iterations and uniformly samples one intermediate iteration $d \in \{K_{\min}, \dots, K_{\max}-1\}$. The corresponding representation h_d is decoded by the same magnitude-mask and phase decoders used for the final iteration and is directly supervised by the complete enhancement objective.

For compact notation, the enhancement objective at iteration k is defined as

$$\begin{aligned} \mathcal{L}_{\text{enh}}^{(k)} = & 0.9\mathcal{L}_{\text{mag}}^{(k)} + 0.3\mathcal{L}_{\text{pha}}^{(k)} + 0.1\mathcal{L}_{\text{com}}^{(k)} \\ & + 0.1\mathcal{L}_{\text{stft}}^{(k)} + 0.05\mathcal{L}_{\text{metric}}^{(k)} + 0.2\mathcal{L}_{\text{time}}^{(k)}, \end{aligned} \quad (11)$$

where $\mathcal{L}_{\text{stft}}$, $\mathcal{L}_{\text{metric}}$, and $\mathcal{L}_{\text{time}}$ denote the multi-resolution STFT, metric-based perceptual, and time-domain reconstruction losses, respectively. The final and intermediate supervision terms are $\mathcal{L}_{\text{MP}} = \mathcal{L}_{\text{enh}}^{(K_{\max})}$ and $\mathcal{L}_{\text{EIS}} = \mathcal{L}_{\text{enh}}^{(d)}$, respectively. The deepest iteration is excluded from EIS sampling because it is already optimized by $\mathcal{L}_{\text{MP}}$.

The overall training objective is

$$\mathcal{L} = \mathcal{L}_{\text{MP}} + \lambda_{\text{EIS}}\mathcal{L}_{\text{EIS}} + \lambda_{\text{bal}}\mathcal{L}_{\text{bal}} + \lambda_{\text{ADC}}\mathcal{L}_{\text{ADC}}, \quad (12)$$

where $\lambda_{\text{EIS}} = 0.1$, $\lambda_{\text{bal}} = 2.5 \times 10^{-4}$, and $\lambda_{\text{ADC}} = 0.05$.

Training is conducted in two stages. An initial 40-epoch warm-up stage optimizes the enhancement, intermediate-supervision, and routing-balancing objectives without $\mathcal{L}_{\text{ADC}}$. The complete objective in Eq. (12) is subsequently used for joint optimization. All $K_{\max} = 5$ refinement iterations are unrolled during training, whereas hard early exit is enabled only at inference.

3. EXPERIMENTS AND RESULTS

3.1. Experimental Setup

We evaluate the proposed ADER framework on the standard VCTK-DEMAND corpus [19, 20], using its 824-utterance test set. All audio is resampled to 16 kHz. STFT analysis uses a 400-sample window with a 100-sample hop. The dense channel dimension is set to 64 and the dense encoder depth to $D = 2$. For ADER, the minimum and maximum refinement depths are set to $K_{\min} = 2$ and $K_{\max} = 5$, respectively. We use four expert adapters, each with a bottleneck channel dimension of $C/4$.

All models are trained for 100 epochs using AdamW with a learning rate of 5×10^{-4} , $\beta_1 = 0.8$, $\beta_2 = 0.99$, with a global batch size of 4. The CER uses straight-through Gumbel Softmax with hard Top-1 selection during training and deterministic Top-1 routing during inference. The ADC objective is activated after an initial 40-epoch warm-up stage.

We report wideband PESQ (WB-PESQ) [21], STOI [22, 23], segmental SNR (SSNR), and the composite measures CSIG (signal distortion), CBAK (background noise intrusiveness), and COVL (overall quality) [24]. Model complexity is measured by the number of parameters and the mean multiply-accumulate operations per second of input audio (MACs/s). For ADER, the MAC count is computed from the refinement iterations actually executed for each utterance and then averaged over the test set.

3.2. Overall Comparison

Table 1 compares ADER with the original MP-SENet, its reduced-depth MP-SENet*, and the fixed shared-block baseline. ADER contains 0.67 M parameters, reducing the parameter count of MP-SENet by 70.4%. With $\tau = 0.20$, ADER executes 2.33 refinement iterations on average and requires 21.99 GMACs/s, corresponding to a

Table 1. Overall comparison on VCTK-DEMAND. MP-SENet* denotes the reduced-depth MP-SENet with $D = 2$. D denotes the dense encoder (and decoder) depth, while $\bar{N}$ denotes the mean number of executed refinement iterations for block-reuse models. **BOLD** indicates the best score in each metric.

Method	D	$\bar{N}$	Param. (M)	GMACs/s	WB-PESQ	STOI (%)	SSNR	CSIG	CBAK	COVL
MP-SENet [18]	4	4.00	2.26	42.59	3.50	96.15	10.64	4.73	3.95	4.22
MP-SENet* [18]	2	4.00	1.75 ($\downarrow 22.6\%$)	31.43 ($\downarrow 26.2\%$)	3.47 ($\downarrow 0.9\%$)	96.05	10.60	4.71	3.93	4.20
Fixed Shared-Block	2	3.00	0.65 ($\downarrow 71.2\%$)	25.02 ($\downarrow 41.3\%$)	3.46 ($\downarrow 1.1\%$)	95.85	10.52	4.70	3.92	4.19
ADER ($\tau = 0.20$)	2	2.33	0.67 ($\downarrow 70.4\%$)	21.99 ($\downarrow 48.4\%$)	3.39 ($\downarrow 3.1\%$)	95.71	10.34	4.69	3.88	4.14
ADER ($\tau = 0.50$)	2	2.15	0.67 ($\downarrow 70.4\%$)	20.73 ($\downarrow 51.3\%$)	3.37 ($\downarrow 3.7\%$)	95.67	10.30	4.68	3.87	4.13

Table 2. Component analysis of ADC, CER, and EIS. Adaptive variants use $\tau = 0.50$. **BOLD** indicates the best score in each metric.

ADC	CER	EIS	$\bar{N}$	Param. (M)	GMACs/s	WB-PESQ	STOI (%)
–	–	–	3.00	0.65	25.02	3.46	95.85
✓	–	–	2.92	0.66	25.95	3.32	95.74
–	✓	–	3.00	0.67	25.19	3.43	95.81
✓	✓	–	2.93	0.67	26.71	3.40	95.77
✓	✓	✓	2.15	0.67	20.73	3.37	95.67

48.4% reduction in computation relative to MP-SENet. Increasing the threshold to $\tau = 0.50$ further reduces the mean refinement depth to 2.15 and the average computation to 20.73 GMACs/s, a 51.3% reduction relative to MP-SENet.

Compared with the fixed shared-block model at $\bar{N} = 3$, ADER trades a moderate decrease in enhancement quality for lower average computation. At $\tau = 0.20$, the computational cost decreases from 25.02 to 21.99 GMACs/s, a reduction of 12.1%, while WB-PESQ decreases by about 2.0% from 3.46 to 3.39. With $\tau = 0.50$, the computation is further reduced by 17.1% to 20.73 GMACs/s, with a corresponding 2.6% decrease in WB-PESQ. Unlike the fixed-depth model, ADER selects the executed refinement depth for each utterance, allowing the average computational cost to be adjusted through the continuation threshold.

3.3. Component Analysis

Table 2 further examines the contributions of ADC, CER, and EIS. Introducing ADC alone reduces WB-PESQ from 3.46 to 3.32, indicating that direct early termination makes intermediate representations a major source of quality degradation. CER alone causes a smaller reduction, yielding a WB-PESQ of 3.43. When CER is combined with ADC, WB-PESQ increases from 3.32 to 3.40, showing that conditional expert routing improves the adaptive-depth configuration without changing its mean refinement depth substantially.

Adding EIS shifts the adaptive model toward substantially shallower execution. The mean refinement depth decreases from 2.93 to 2.15, while the average computation decreases from 26.71 to 20.73 GMACs/s. Meanwhile, WB-PESQ changes from 3.40 to 3.37. These results indicate that direct supervision of intermediate outputs enables the ADC to select earlier exits with a limited additional quality loss.

3.4. Refinement Depth and Adaptive Computation

Figure 2(a) evaluates the EIS-trained model with ADC disabled and a fixed refinement depth for all test utterances. WB-PESQ increases from 3.36 at $\bar{N} = 2$ to 3.43 at $\bar{N} = 3$, while further refinement produces little additional improvement. This result indicates that the perceptual benefit of progressive refinement largely saturates after

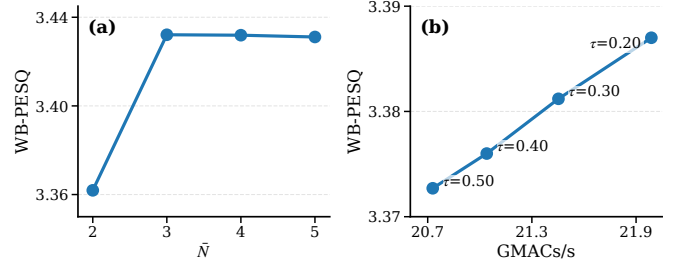


Fig. 2. Analysis of refinement depth and adaptive computation in ADER. (a) WB-PESQ under different forced depths $\bar{N}$. (b) Computation–quality trade-off under different ADC continuation threshold τ .

the third iteration, supporting the use of input dependent termination rather than a fixed maximum depth for every utterance.

Figure 2(b) shows the computation–quality trade-off obtained by varying the ADC continuation threshold without changing the model parameters. Increasing τ from 0.20 to 0.50 reduces the mean refinement depth from 2.33 to 2.15 and the average computation from 21.99 to 20.73 GMACs/s, while WB-PESQ decreases from 3.387 to 3.372. Thus, the continuation threshold provides a direct control over the average inference cost through hard input-dependent termination.

4. CONCLUSION

We presented ADER for efficient SE with parameter-shared progressive refinement and input-dependent execution. By combining ADC, CER, and EIS, ADER supports hard early termination while retaining a compact shared refinement backbone. Experiments on VCTK-DEMAND show that ADER substantially reduces model size and average computation, with the continuation threshold providing a controllable trade-off between computational cost and enhancement quality. Forced-depth evaluation further shows limited WB-PESQ gains beyond the third refinement iteration, while the component analysis demonstrates the complementary roles of CER and EIS under adaptive-depth inference. These results show that adaptive block reuse can reduce redundant computation in SE. The current evaluation is limited to VCTK-DEMAND and an MP-SENet backbone. Future work will examine ADER on broader datasets and other SE architectures to assess its general applicability.

5. REFERENCES

- [1] Yanxin Hu, Yun Liu, Shubo Lv, Mengtao Xing, Shimin Zhang, Yihui Fu, Jian Wu, Bihong Zhang, and Lei Xie, “DCCRN:

- Deep complex convolution recurrent network for phase-aware speech enhancement,” in *Proc. Interspeech*. 2020, pp. 2472–2476, ISCA.
- [2] Dacheng Yin, Chong Luo, Zhiwei Xiong, and Wenjun Zeng, “PHASEN: A phase-and-harmonics-aware speech enhancement network,” in *Proc. AAAI Conf. Artificial Intelligence*. 2020, pp. 9458–9465, AAAI Press.
 - [3] Szu-Wei Fu, Cheng Yu, Tsun-An Hsieh, Peter Plantinga, Mirco Ravanelli, Xugang Lu, and Yu Tsao, “MetricGAN+: An improved version of MetricGAN for speech enhancement,” in *Proc. Interspeech*. 2021, pp. 201–205, ISCA.
 - [4] Sherif Abdulatif, Ruizhe Cao, and Bin Yang, “CMGAN: Conformer-based metric-GAN for monaural speech enhancement,” *IEEE ACM Trans. Audio Speech Lang. Process.*, vol. 32, pp. 2477–2493, 2024.
 - [5] Ye-Xin Lu, Yang Ai, and Zhen-Hua Ling, “Explicit estimation of magnitude and phase spectra in parallel for high-quality speech enhancement,” *Neural Networks*, vol. 189, pp. 107562, 2025.
 - [6] Wangyou Zhang, Kohei Saijo, Jee-weon Jung, Chenda Li, Shinji Watanabe, and Yanmin Qian, “Beyond performance plateaus: A comprehensive study on scalability in speech enhancement,” in *Proc. Interspeech*. 2024, ISCA.
 - [7] Xiang Hao, Xiangdong Su, Radu Horaud, and Xiaofei Li, “FullSubNet: A full-band and sub-band fusion model for real-time single-channel speech enhancement,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2021, pp. 6633–6637, IEEE.
 - [8] Hendrik Schröter, Alberto N. Escalante-B., Tobias Rosenkranz, and Andreas Maier, “DeepFilterNet: A low complexity speech enhancement framework for full-band audio based on deep filtering,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2022, pp. 7407–7411, IEEE.
 - [9] Sunwoo Kim and Minje Kim, “Bloom-Net: Blockwise optimization for masking networks toward scalable and efficient speech enhancement,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2022, pp. 366–370, IEEE.
 - [10] Jangyeon Kim, Ui-Hyeop Shin, Jaehyun Ko, and Hyung-Min Park, “Stack less, repeat more: A block reusing approach for progressive speech enhancement,” in *Proc. Interspeech*. 2025, ISCA.
 - [11] Alex Graves, “Adaptive computation time for recurrent neural networks,” *arXiv preprint arXiv:1603.08983*, 2016.
 - [12] Andong Li, Chengshi Zheng, Lu Zhang, and Xiaodong Li, “Learning to inference with early exit in the progressive speech enhancement,” in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*. 2021, pp. 466–470, IEEE.
 - [13] Riccardo Miccini, Alaa Zniber, Clément Laroche, Tobias Piechowiak, Martin Schoeberl, Luca Pezzarossa, Ouassim Karakchou, Jens Sparsø, and Mounir Ghogho, “Dynamic nsNET2: Efficient deep noise suppression with early exiting,” in *Proc. IEEE Int. Workshop Mach. Learn. Signal Process. (MLSP)*. 2023, pp. 1–6, IEEE.
 - [14] Sanyuan Chen, Yu Wu, Zhuo Chen, Takuya Yoshioka, Shujie Liu, Jin-Yu Li, and Xiangzhan Yu, “Don’t shoot butterfly with rifles: Multi-channel continuous speech separation with early exit transformer,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2021, pp. 6139–6143, IEEE.
 - [15] Kenny Olsen, Mads Østergaard, Karl Ulbæk, Søren Føns Nielsen, Rasmus Malik Høegh Lindrup, Bjørn Jensen, and Morten Mørup, “Knowing when to quit: Probabilistic early exits for speech separation networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*. 2026, vol. 2026, pp. 104332–104352, OpenReview.net.
 - [16] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. Int. Conf. Learning Representations (ICLR)*. 2017, OpenReview.net.
 - [17] William Fedus, Barret Zoph, and Noam Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, pp. 120:1–120:39, 2022.
 - [18] Ye-Xin Lu, Yang Ai, and Zhen-Hua Ling, “MP-SENet: A speech enhancement model with parallel denoising of magnitude and phase spectra,” in *Proc. Interspeech*. 2023, pp. 3834–3838, ISCA.
 - [19] Cassia Valentini-Botinhao, “Noisy speech database for training speech enhancement algorithms and TTS models,” University of Edinburgh, School of Informatics, Centre for Speech Technology Research (CSTR), 2017.
 - [20] Joachim Thiemann, Nobutaka Ito, and Emmanuel Vincent, “The diverse environments multi-channel acoustic noise database (DEMAND): A database of multichannel environmental noise recordings,” *Proceedings of Meetings on Acoustics*, vol. 19, no. 1, pp. 035081, 05 2013.
 - [21] Antony W. Rix, John G. Beerends, Michael P. Hollier, and Andries P. Hekstra, “Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2001, pp. 749–752, IEEE.
 - [22] Cees H. Taal, Richard C. Hendriks, Richard Heusdens, and Jesper Jensen, “A short-time objective intelligibility measure for time-frequency weighted noisy speech,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*. 2010, pp. 4214–4217, IEEE.
 - [23] Cees H. Taal, Richard C. Hendriks, Richard Heusdens, and Jesper Jensen, “An algorithm for intelligibility prediction of time-frequency weighted noisy speech,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.
 - [24] Yi Hu and Philippos C. Loizou, “Evaluation of objective quality measures for speech enhancement,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 16, no. 1, pp. 229–238, 2008.